

Adaptive Computation and Machine Learning

COMS 4030A / COMS 7047A

Mushroom Classification Using Machine Learning

UCI Mushroom Dataset

Name	Student Number
Mashego Mabeloane	2654606
Dolf Shungube	2683067

University of the Witwatersrand

May 2026

Contents

1	Dataset	2
1.1	Source	2
1.2	Description	2
1.3	Objective	2
2	Preprocessing and Data Splits	2
2.1	Data cleaning	2
2.2	Feature Selection	2
2.3	Encoding	3
2.4	Data Split	3
3	Models	4
3.1	Random Forest	4
3.2	Dense Neural Network	4
4	Experiments and Hyperparameter Tuning	5
4.1	Random Forest — max_depth Tuning	5
4.2	Dense Neural-Network — nodes per Layer,	5
5	Graphs	5
5.1	Validation Accuracy vs max_depth — Random Forest	5
5.2	Dense Neural Network Experiments	6
5.2.1	Training Process	6
5.2.2	Setting Dense Layer Size to 1	7
5.2.3	Dense Layer Size: 2	8
5.2.4	Changing Features	10
5.3	Confusion Matrices — Random Forest	12
6	Results	13
7	Conclusion	13

1 Dataset

1.1 Source

The dataset used in this project is the **UCI Mushroom Dataset**, obtained from the UCI Machine Learning Repository: <https://archive.ics.uci.edu/ml/datasets/mushroom>.

1.2 Description

The dataset contains **8,124 instances** and **22 categorical features** describing physical characteristics of mushrooms, such as cap-shape, odor, gill-color, cap-surface, cap-color, bruises, gill-attachment, gill-spacing, gill-size, gill-color, stalk-shape, stalk-root, stalk-surface-above-ring, stalk-surface-below-ring, stalk-color-above-ring, stalk-color-below-ring, veil-type, veil-color, ring-number, ring-type, spore-print-color, population and habitat. The target variable is binary: *edible* (e) or *poisonous* (p). All features are represented as single-character categorical values.

1.3 Objective

The objective of this project is to build and evaluate machine learning models capable of accurately classifying mushrooms as edible or poisonous based on their physical characteristics.

2 Preprocessing and Data Splits

2.1 Data cleaning

For data cleaning we performed the following steps:

- Handling missing values
- Handling duplicates
- Handling inconsistent categories
- Encode categories into numerical representations
- Handle rare categories
- Prepare the data for modeling

2.2 Feature Selection

Chi-square (χ^2) statistical tests were applied to identify features with the strongest association with the target variable. The χ^2 statistic measures the deviation between observed and expected frequencies, while the p-value indicates whether the association is statistically significant. Features were ranked by their χ^2 score and grouped into two tiers:

Table 1: Chi-Square Feature Selection Results

Feature	χ^2 Score	p-value	Tier
odor	7658.72	0.0	Tier 1
spore-print-color	4604.11	0.0	Tier 1
gill-color	3767.53	0.0	Tier 1
ring-type	2958.10	0.0	Tier 1
stalk-surface-above-ring	2809.22	0.0	Tier 1
stalk-surface-below-ring	2685.32	0.0	Tier 1
gill-size	2365.83	0.0	Tier 1
stalk-color-above-ring	2238.37	0.0	Tier 1
stalk-color-below-ring	2152.86	0.0	Tier 1
bruises	2043.02	0.0	Tier 1
population	1930.68	0.0	Tier 1
habitat	1572.99	0.0	Tier 1
stalk-root	1345.76	3.98e-290	Tier 2
gill-spacing	983.86	5.79e-216	Tier 2
cap-shape	489.99	1.16e-103	Tier 2
cap-color	387.75	5.61e-78	Tier 2
ring-number	374.63	4.46e-82	Tier 2
cap-surface	314.77	6.31e-68	Tier 2
veil-color	191.18	3.39e-41	Tier 2
gill-attachment	133.94	5.62e-31	Tier 2
stalk-shape	83.91	5.17e-20	Tier 2
veil-type	0.00	1.0	Dropped

The feature `veil-type` was dropped entirely as it showed no association with the target variable ($\chi^2 = 0$, $p = 1.0$).

2.3 Encoding

All features are categorical string values. Label encoding was applied to convert each feature into numerical integers, as required by both sklearn and Keras models.

2.4 Data Split

The dataset was split into three sets to ensure unbiased evaluation:

Table 2: Random Forest Data Split

Set	Proportion	Purpose
Train	64%	Model learning
Validation	16%	Hyperparameter tuning
Test	20%	Final evaluation

Table 3: Dense-Neural-Network Data Split

Set	Proportion	Purpose
Train	60%	Model learning
Validation	20%	Observing overfitting
Test	20%	Final evaluation

Two experiments were defined using different feature sets for the Random Forest model:

- **Experiment 1** — Tier 1 features only (12 features)
- **Experiment 2** — Tier 1 + Tier 2 features (21 features)

3 Models

3.1 Random Forest

A Random Forest was used for this project to serves as the classical baseline model.

3.2 Dense Neural Network

A Multilayer Perceptron was implemented using Keras as the deep learning model. The architecture is as follows:

Table 4: NN Architecture

Layer	nodes	Activation
Input	12	—
Embeddings		
Concatenation		
Dense	2	ReLU
Dropout	0.3	—
Dense	1	ReLU
Dropout	0.3	—
Dense (output)	1	Sigmoid

- Each node in the Input Layer represents a separate feature.
- At the embedding layer each feature receives its own separate embedding, used to give meaning that can be learned by the neural network.
- The concatenation layer concatenates all the separate embedding into 1 vector so the dense layer can see how they interact.
- Each Dense layer is fully connected with layers it lies in between of.
- The dropout layer works to set some weights for the features to 0 as a way of adding regularization to avoid overfitting.

The model was compiled with the `adam` optimizer, `binary_crossentropy` loss, and `accuracy` as the metric.

4 Experiments and Hyperparameter Tuning

4.1 Random Forest — `max_depth` Tuning

The `max_depth` hyperparameter was tuned across values `{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 15, 20, None}` with `n_estimators=100`. Results are presented in Section 5.

4.2 Dense Neural-Network — `nodes per Layer`,

The dropout rate at each dense layer was set to 0.3 for all models, this value was not adjusted because the performance of the model mainly depends on the number of nodes in each later. The results presented in Section 5, show the results of changing the number of nodes in each layer of the model. The results then show the affect of changing the number of features the model trains on.

.

5 Graphs

5.1 Validation Accuracy vs `max_depth` — Random Forest

The figure below presents the validation accuracy of the Random Forest model as a function of `max_depth` for both feature configurations. Both experiments converge to 100% accuracy as tree depth increases, however the Tier 1 feature set converges faster, achieving higher accuracy at shallower depths. Notably, at `max_depth = 2` and `3`, the Tier 1 + Tier 2 configuration performs slightly worse than Tier 1 alone, suggesting that the additional Tier 2 features introduce mild noise at lower tree depths, requiring deeper trees to compensate. Beyond `max_depth = 8`, both configurations achieve identical perfect accuracy, further confirming that the Tier 2 features contribute no additional discriminative power over the Tier 1 features alone.

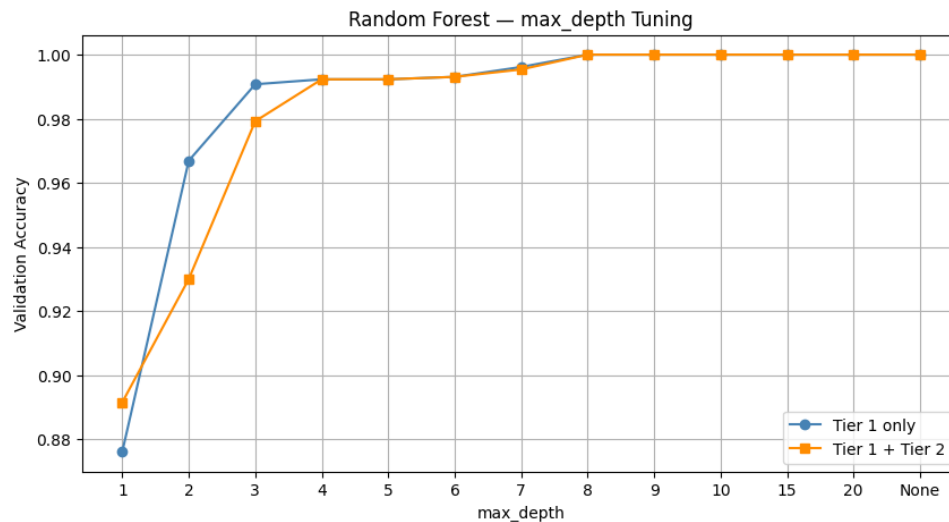


Figure 1: Random Forest — Validation Accuracy vs max_depth (Tier 1 vs Tier 1+2)

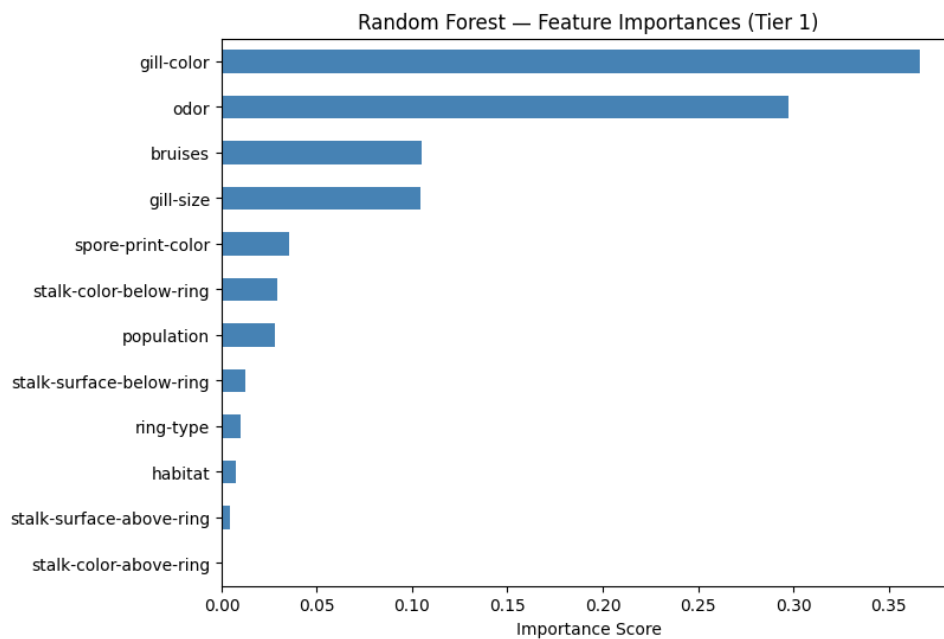


Figure 2: Random Forest — Feature Importance (Tier 1)

5.2 Dense Neural Network Experiments

5.2.1 Training Process

- Before training can begin the entries corresponding to the feature being modeled were isolated from the complete data set.
- Then the isolated data was split into a training set, evaluation set and testing set.
- The training was done on the training set
- At the end of each epoch, the performance of the model was measured with the validation set and the results recorded.

- Each model was trained for an epoch count of 10 or 15. item A batch size of 30 was used in updating the model parameters.

5.2.2 Setting Dense Layer Size to 1

In this experiment the model was allowed to train for an epoch count of 10, with the 0.3 dropout rate using the Tier 1 features only.

The graph below shows the training and validation accuracy when the number of nodes in each dense layer is set to 1. The value of both the training and validation accuracy remain constant indicating that the model does not improve in this case. The poor performance of this model is further supported by the training-validation-loss plot, which indicate that the loss decreased by a negligible value.

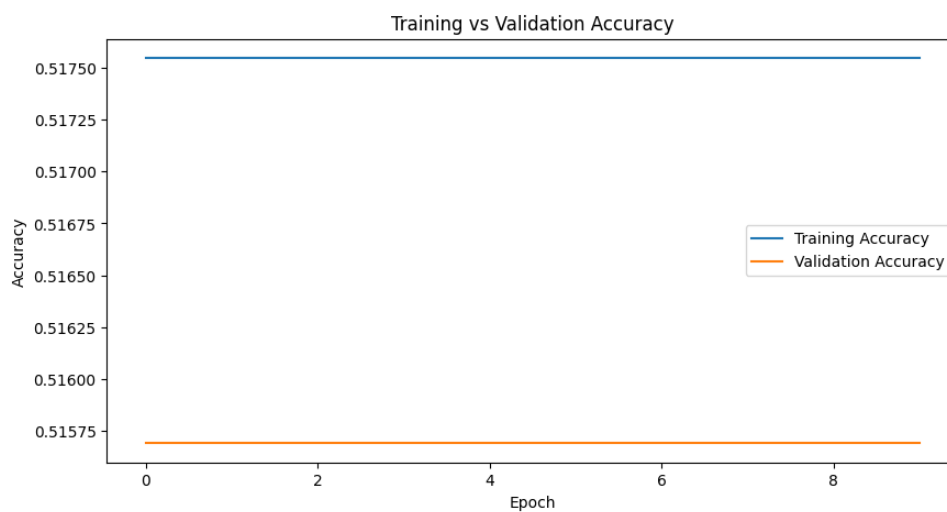


Figure 3: Training and Validation Accuracy: layer size=1

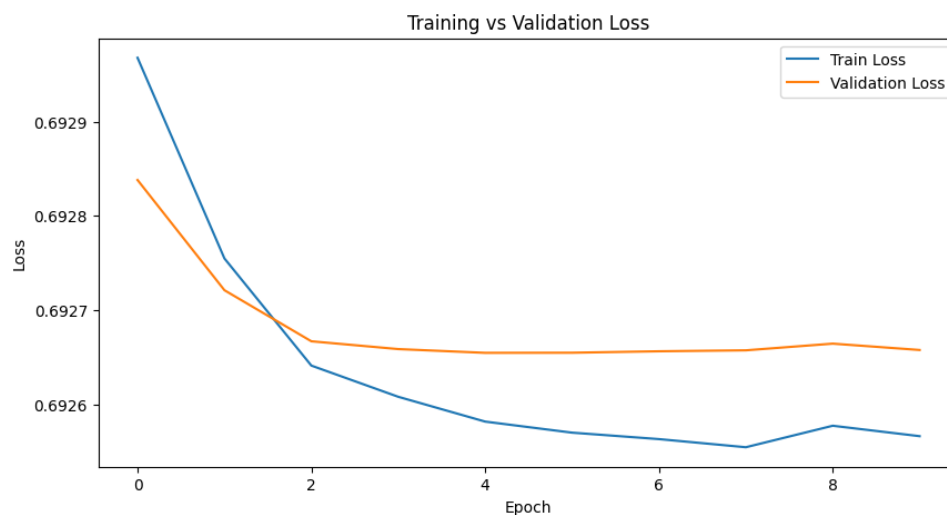


Figure 4: Training and Validation: layer- size=1

The simplicity of the model prevents it from learning more complex relationships. The confusion matrices below confirm this statement, considering that the number edible

mushroom entries are slightly over 50% of the data , this model only learned to output only the "edible" label, considering that the data is balanced, the model would be correct 50% of the time from always outputting "edible".

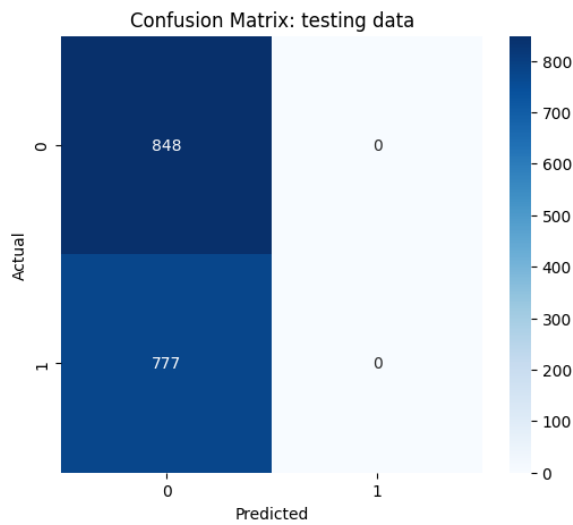


Figure 5: Confusion Matrix — Testing set (Tier 1)

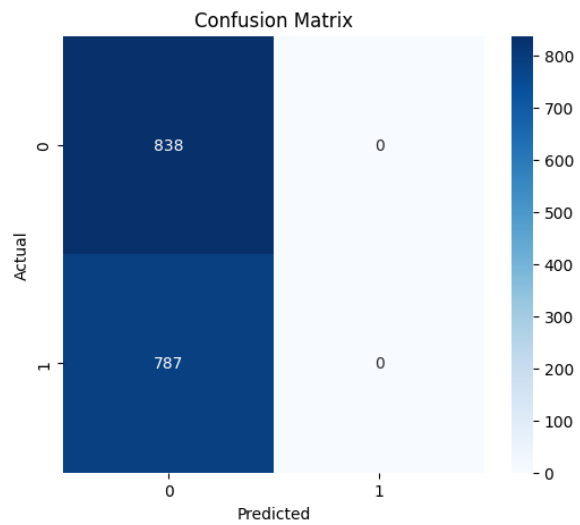


Figure 6: Confusion Matrix — Validation Set(Tier 1)

5.2.3 Dense Layer Size: 2

In this experiment the model was allowed to train for an epoch count of 10, with the 0.3 dropout rate using the Tier 1 features only. The number of nodes for the first dense layer was increased to 2, and the number of nodes in the second dense layer remained 1.

The graph below shows the training-validation accuracy when the number of nodes in the first dense layers is set to 2. There is a clear improvement in the performance of the model to when compared to the initial single node dense layer model. The model maintained a high validation-accuracy during training, close to 1 indicating that with the initial training is was already capable of predicting most of the output of the validation set. At the highest level The model maintained a training accuracy of about 0.8, which is far higher than the initial model.

Compared to the initial model, the training-validation loss function supports that this agent was improving with higher decreasing in the training loss.

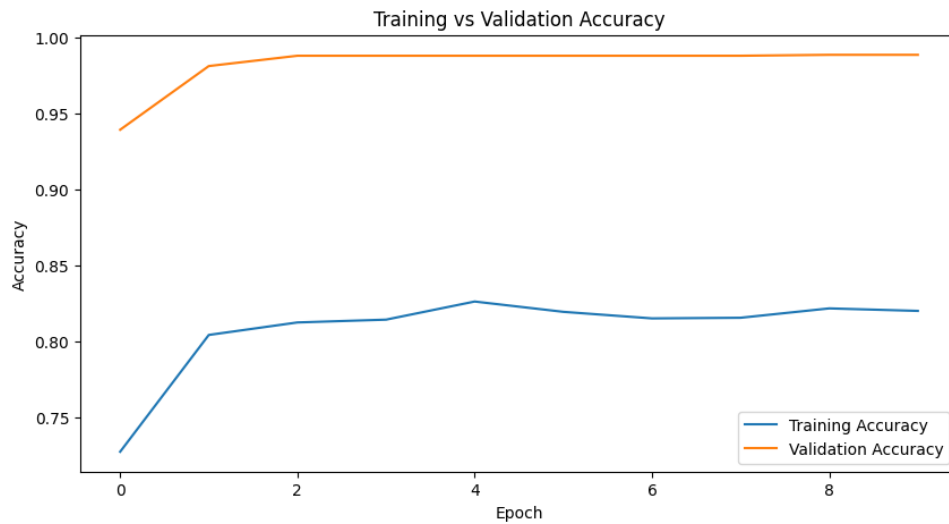


Figure 7: Training and Validation Accuracy: layer size=2

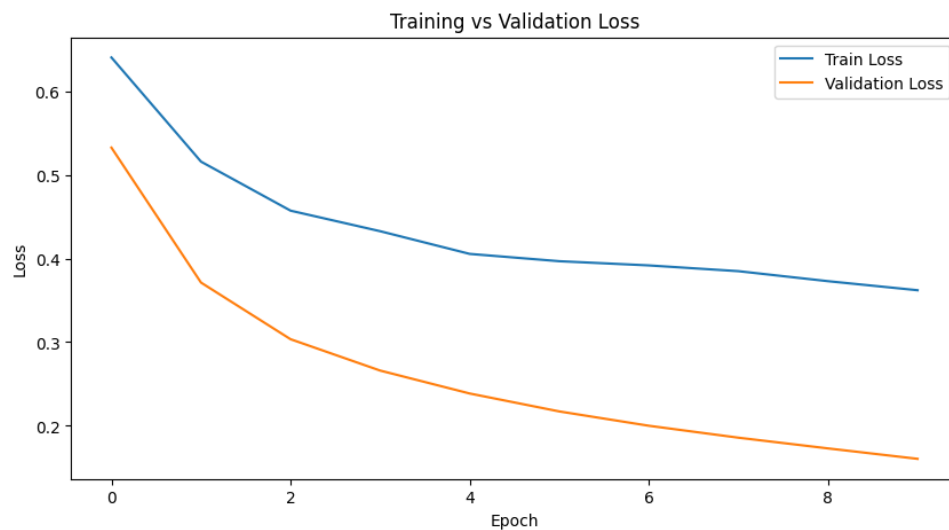


Figure 8: Training and Validation Loss: layer=2

This model was able to learn more subtle relationships between the features only misclassifying 15 entries in the testing set , 19 entries in the validation set, and 46 entries in the training set, achieving a 0.99 accuracy in all data sets.

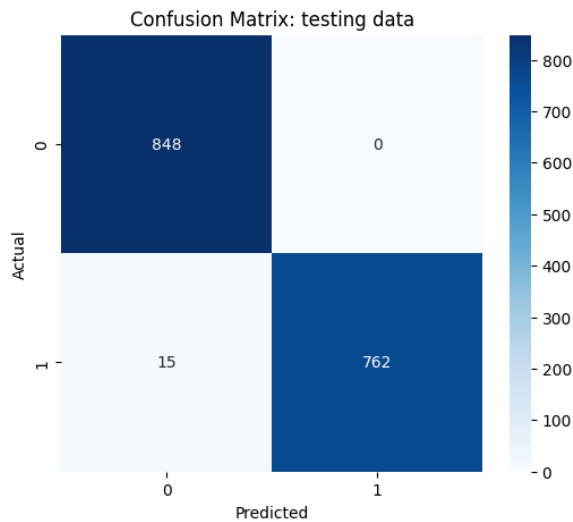


Figure 9: Confusion Matrix — Testing set (Tier 1)

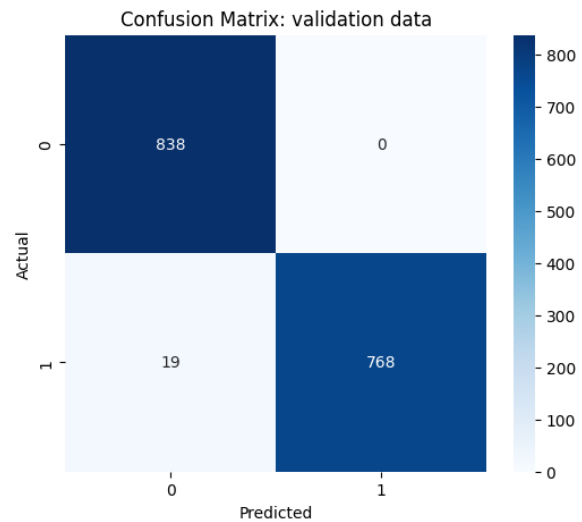


Figure 10: Confusion Matrix — Validation Set(Tier 1)



Figure 11: Confusion Matrix — Validation Set(Tier 1)

5.2.4 Changing Features

The great performance of the model with only a few nodes in the network showed that it might be possible to get a strong model with fewer features. In this experiment the model was trained only from the "odor" feature, this was done because feature selection showed that it is the most informative feature.

For this session validation accuracy remained High, close to 1 and the training accuracy stayed at around 0.8, showing that the model still performs great even with a decrease in the number of features. This value is slower than that of the 2-node model, suggesting that reducing the number of features means there are some interactions this model is not able to learn.

When Compared to the model with 2 nodes in each dense layer, the training and validation loss are decreasing at a slower rate, this is clearly visible with the testing loss, further

supporting that there are some interactions this model is not able to learn leading to more mis-classifications.

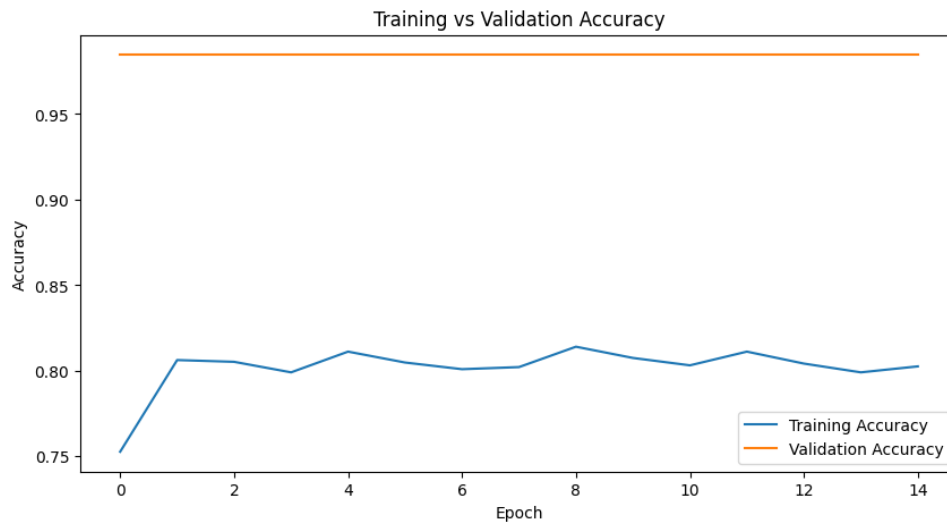


Figure 12: Training and Validation Loss: odor



Figure 13: Training and Validation: odor

The Confusion matrices below show that simplifying the model by training on fewer features does decrease its accuracy, more points where mis-classified to when compared with the model that uses all the Tier 1 features for training. The decrease in accuracy is small, with the accuracy decreasing from 0.99 to 0.98 for all the sets, showing that for most mushrooms odor can be good enough to decide on whether it is edible and the mis-classifications show that using odor alone to decide on the edibility of mushrooms does not work for all mushrooms.

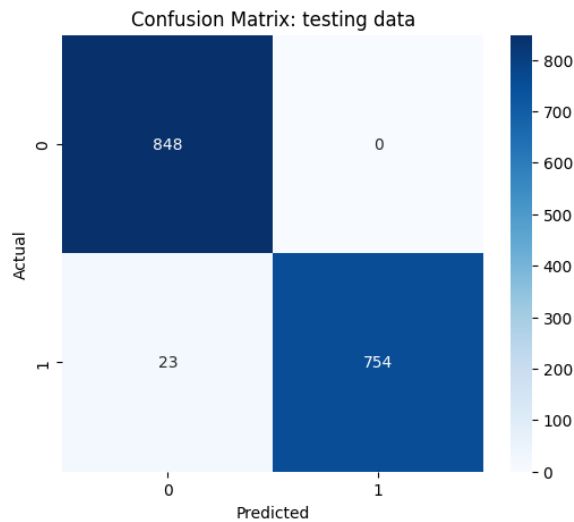


Figure 14: Confusion Matrix — Testing set: odor

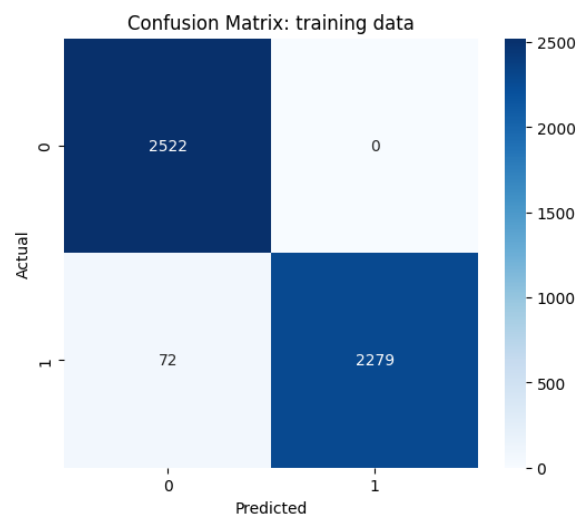


Figure 15: Confusion Matrix — Training Set: odor

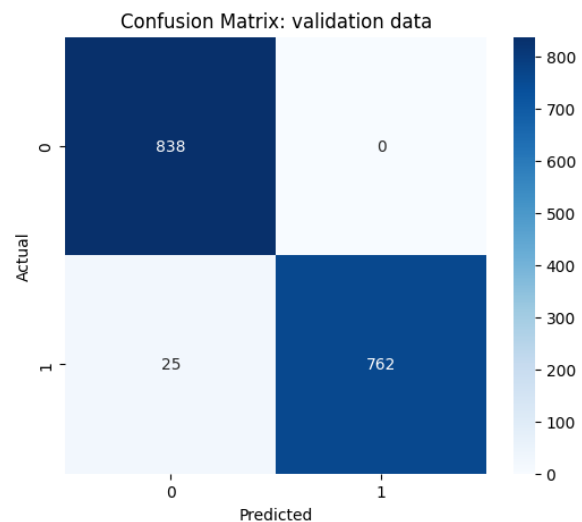


Figure 16: Confusion Matrix — Validation Set: odor

5.3 Confusion Matrices — Random Forest

The confusion matrices below present the classification results of the Random Forest model on the test set for both feature configurations at `max_depth = 20`. In both experiments, the model correctly classified all instances with no false positives or false negatives, achieving perfect precision and recall for both the edible and poisonous classes. The identical results across both configurations further confirm that the Tier 2 features provide no additional discriminative power beyond what is already captured by the Tier 1 features.

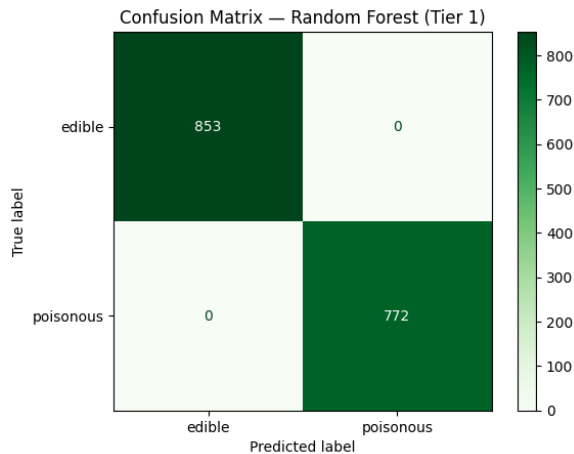


Figure 17: Confusion Matrix — Random Forest (Tier 1)

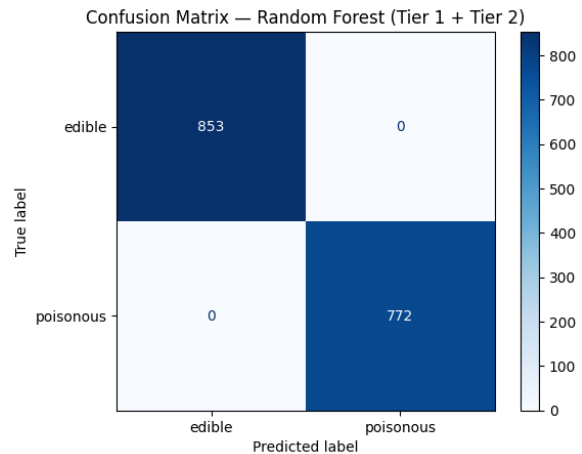


Figure 18: Confusion Matrix — Random Forest (Tier 1+2)

Figure 19: Confusion Matrix — MLP (Tier 1)

6 Results

Table 5: Test Set Results — All Models

Model	Features	Accuracy	Precision	Recall	F1
Random Forest	Tier 1	1.0	1.0	1.0	1.0
Random Forest	Tier 1+2	1.0	1.0	1.0	1.0
DNN	Tier 1	0.99	0.99	0.99	0.99
DNN	odor	0.98	0.98	0.99	0.99

7 Conclusion

The chi-square feature selection process successfully identified the most predictive features in the dataset. Both the Random Forest and DNN models were evaluated across two feature sets. The results demonstrated that a single feature can be used to decide on the edibility of many mushrooms but not all, some other classifications need multiple features to decide on this. All the chosen models have high accuracy thus any of them are good enough to used in classification of mushrooms as edible or poisonous.

References

- [1] UCI Machine Learning Repository. *Mushroom Dataset*. <https://archive.ics.uci.edu/ml/datasets/mushroom>
- [2] Pedregosa et al. *Scikit-learn: Machine Learning in Python*. JMLR 12, pp. 2825–2830, 2011.

[3] Chollet et al. *Keras*. <https://keras.io>, 2015.